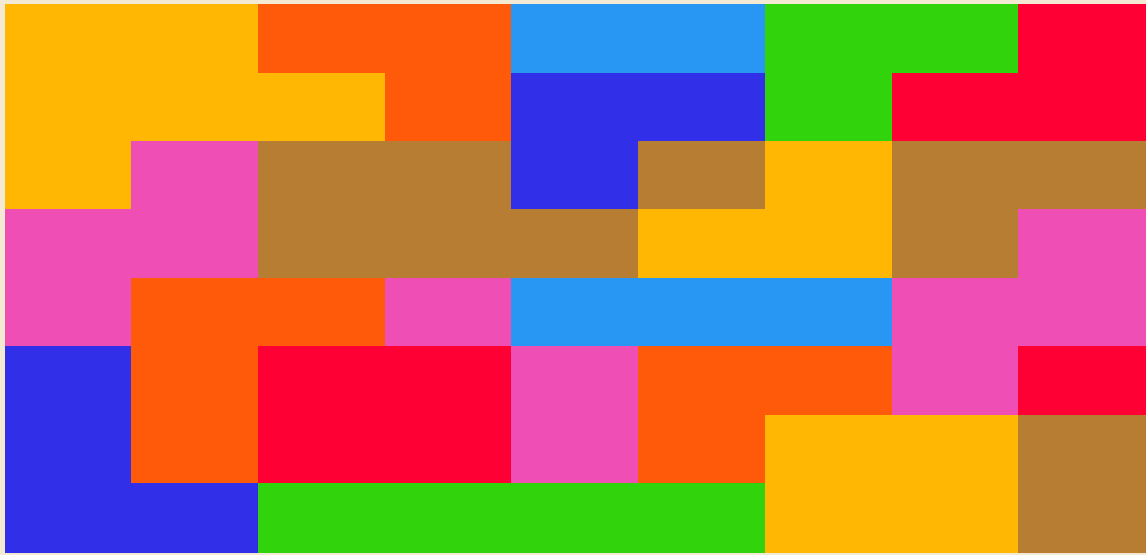




DRONE CAMP

DAY 3

Loops, decisions, and reusable flight routines.

PYTHON + EASYTELLO + CONTROL FLOW

MISSION

Make the flight plan think

On Day 2, a Python file described one fixed sequence of EasyTello commands. Day 3 adds control flow: rules that decide which statements execute, how many times they execute, and which named routine should run. Python still reads from top to bottom, but a loop can return to an earlier block, a conditional can skip one branch and choose another, and a function can package several commands under one useful name.

The central question

Which part of the program controls repetition, which part controls decisions, and which part gives a flight routine a reusable name?

Three tools, three jobs

- for loop: repeat a known pattern a known number of times
- conditional: choose a branch after testing a true-or-false expression
- function: group related instructions and allow values to enter through parameters

PROGRAM MAP

```
Python control flow
|
+-- for loop -> repeat commands
|
+-- if/elif/else -> choose a route
|
|-- function -> name and reuse a routine
      |
      v
    EasyTello methods
      |
      v
    Tello command messages
```

Python decides what should happen. EasyTello translates the selected method calls into commands that the drone understands.

CHECKPOINT 01

Repeat a movement pattern with a for loop

FILE LEVEL - A FOUR-SIDE ROUTE

```
from easytello import tello

drone = tello.Tello()

drone.takeoff()
drone.up(30)

for side in range(4):
    drone.forward(40)
    drone.cw(90)
    drone.wait(1)

drone.land()
drone.close()
```

What range(4) contributes

range(4) provides four loop cycles. During those cycles, side receives 0, then 1, then 2, then 3. The program does not need to display those values; Python still uses them to count the repetitions. The indented block belongs to the loop, so all three method calls repeat together.

Why this pattern suggests a square

Each cycle moves forward once and rotates ninety degrees once. Repeating that pair four times creates four sides and four turns. Four ninety-degree turns total 360 degrees, so the intended final heading matches the starting heading. Real flight may drift slightly, which is why the route should be short and the flight area should remain clear.

Trace first

Write the expected position and heading after each cycle before the propellers begin moving.

CHECKPOINT 02

Choose a branch with a conditional

FILE LEVEL - IF/ELSE CHOOSES WHETHER TO FLY

```
from easytello import tello

MIN_BATTERY = 40

drone = tello.Tello()
battery = int(drone.get_battery())

print(f"Battery: {battery}%")

if battery < MIN_BATTERY:
    print("Battery is below the lesson threshold.")
    print("Charge the drone before flying.")
else:
    print("Battery check passed.")
    drone.takeoff()
    drone.wait(2)
    drone.land()

drone.close()
```

The expression becomes True or False

The condition `battery < MIN_BATTERY` compares two integers. Python evaluates that comparison before choosing a branch. When the expression is True, only the first indented block runs. When it is False, Python skips that block and runs the else block instead. The branches are mutually exclusive during one execution.

Why convert the response with `int()`?

`get_battery()` returns the drone's response. `int(...)` converts a numeric response into an integer so Python can compare it with 40. The threshold here is a classroom rule for this exercise, not a replacement for the drone's official operating guidance or an instructor's preflight check.

Decision point

A conditional does not move the drone by itself. It decides which EasyTello method calls Python will reach.

CHECKPOINT 03

Give a flight routine a function name

FUNCTION DEFINITION -> FUNCTION CALL

```
def fly_square(drone, distance, angle):  
    drone.takeoff()  
    drone.up(30)  
  
    for side in range(4):  
        drone.forward(distance)  
        drone.cw(angle)  
        drone.wait(1)  
  
    drone.land()  
  
from easytello import tello  
  
my_drone = tello.Tello()  
fly_square(my_drone, 40, 90)  
my_drone.close()
```

Definition and call are different moments

The def statement teaches Python what fly_square means. The indented body is stored as a reusable routine, but it does not execute at the moment Python reads the definition. The later function call starts the routine and supplies three arguments: the drone object, a distance, and an angle.

Parameters make the routine adjustable

Inside the function, drone, distance, and angle are parameters. They receive the arguments supplied by the call. Because the routine uses those names instead of fixed values, another call can reuse the same logic with a different distance or turn angle. The algorithm remains recognizable while its inputs change.

Read it aloud

Define a routine named fly_square that needs a drone, a distance, and an angle.

CHECKPOINT 04

Map decisions to named flight routines

FUNCTIONS DEFINE ROUTES - CONDITIONAL SELECTS ONE

```
def fly_line(drone, distance):
    drone.takeoff()
    drone.up(30)
    drone.forward(distance)
    drone.wait(1)
    drone.land()

def fly_square(drone, distance):
    drone.takeoff()
    drone.up(30)

    for side in range(4):
        drone.forward(distance)
        drone.cw(90)
        drone.wait(1)

    drone.land()

if battery >= 60:
    fly_square(drone, 40)
elif battery >= 40:
    fly_line(drone, 40)
else:
    print("Charge the drone before flying.")
```

How if, elif, and else are tested

Python checks the conditions from top to bottom. A battery value of 72 satisfies the first comparison, so fly_square runs and the remaining branches are skipped. A value of 48 fails the first comparison but satisfies the elif comparison, so fly_line runs. A lower value reaches the else branch. Exactly one branch is selected.

Why functions belong at the branch endpoints

The conditional remains readable because each branch names a complete behavior instead of containing every movement command directly. The decision answers which route should run; the selected function answers how that route works. Separating those responsibilities makes both the logic and the movement sequence easier to inspect.

Design habit

Use conditionals to choose behavior and functions to describe behavior.

FINAL BUILD

Combine loops, decisions, functions, and EasyTello

COMPLETE DAY 3 PROGRAM

```
from easytello import tello

MIN_BATTERY = 40
MOVE_DISTANCE = 40
TURN_ANGLE = 90

def battery_ready(drone):
    battery = int(drone.get_battery())
    print(f"Battery: {battery}%")
    return battery >= MIN_BATTERY

def fly_square(drone, distance, angle):
    drone.takeoff()
    drone.up(30)

    for side in range(4):
        drone.forward(distance)
        drone.cw(angle)
        drone.wait(1)

    drone.land()

def run_mission():
    drone = tello.Tello()

    if battery_ready(drone):
        fly_square(
            drone,
            MOVE_DISTANCE,
            TURN_ANGLE
        )
    else:
        print("Charge the drone before flying.")

    drone.close()

run_mission()
```

Follow the values through the program

run_mission() creates the EasyTello object. battery_ready(drone) requests a value and returns True or False. The if statement uses that return value to choose whether fly_square(...) runs. Inside the selected routine, the loop repeats the forward-turn pattern. Every movement still reaches the drone through an EasyTello method.

Before launch

Trace the call order: run_mission -> battery_ready -> conditional -> fly_square -> EasyTello methods.

FLIGHT TOOLKIT

Build shapes with straight edges and turns

A polygon route does not require a curved-flight command. Each side is a straight movement, and each corner is a rotation. The drone travels forward relative to the direction its front is currently facing. After a turn, the next forward command therefore begins a new edge in a new direction. For this camp setup, `curve()` is intentionally excluded because it is not functioning reliably.

EASYTELLO METHODS USED BY THE ASSIGNMENTS

```
drone.takeoff()      # begin the flight
drone.up(30)         # create safe working height
drone.forward(distance) # draw one straight edge
drone.cw(angle)       # turn right
drone.ccw(angle)      # turn left
drone.wait(1)         # pause between commands
drone.land()          # finish normally
drone.close()         # close the local socket
```

Calculate the corner turn

For a regular polygon, use the exterior turn: 360 divided by the number of sides. A triangle uses 120 degrees, a square uses 90 degrees, and a pentagon uses 72 degrees. A rectangle also uses ninety-degree turns, but it alternates between a long distance and a short distance.

Stage	Requirement	Evidence
Triangle	3 equal edges; turn 120 degrees after each edge	3 movement cycles
Rectangle	4 edges; alternate long and short distances; turn 90 degrees	2 long + 2 short
Pentagon	5 equal edges; turn 72 degrees after each edge	5 movement cycles

Work while waiting

Complete the route trace, starter-code blanks, and extension tasks before requesting access to a drone.

ASSIGNMENT 01

Write a triangle flight function

Your first function should accept a drone object and one side length. The three edges are equal, so a for loop can repeat one forward command and one turn. Before writing Python, calculate the total rotation and predict which direction the drone should face after the third turn.

FUNCTION ASSIGNMENT - COMPLETE THE TWO BLANKS

```
def fly_triangle(drone, side_length):
    drone.takeoff()
    drone.up(30)

    for side in range(__):
        print(f"Flying side {side + 1}")
        drone.forward(side_length)
        drone.cw(__)
        drone.wait(1)

    drone.land()
```

Required stages

Stage	Requirement	Evidence
Plan	Draw three arrows and label the turn after each edge.	Paper route diagram
Code	Complete both blanks and keep every repeated command inside the loop.	Function runs without syntax errors
Trace	List side, heading change, and expected location for all three cycles.	Three-row trace
Explain	State why range(3) and a 120-degree turn describe a triangle.	Two clear sentences

Extension missions

- Change cw(...) to ccw(...) and predict the new direction before testing.
- Add a turn_angle parameter instead of writing 120 directly.
- Create two function calls with different side lengths and compare the planned routes.
- Add a print statement that announces every forward distance and turn angle.

Flight check

Do not launch until the function contains one takeoff, one landing, three forward commands, and three turns.

ASSIGNMENT 02

Use a conditional to build a rectangle

A rectangle still has four corners, but its edges are not all the same length. The loop variable can identify which edge is currently being created. When side is even, use the long distance. When side is odd, use the short distance. The modulo operator gives the remainder after division; side % 2 equals zero for even values.

FUNCTION ASSIGNMENT - LOOP + IF/ELSE

```
def fly_rectangle(drone, long_side, short_side):
    drone.takeoff()
    drone.up(30)

    for side in range(4):
        if side % 2 == 0:
            drone.forward(_____)
        else:
            drone.forward(_____)

    drone.cw(____)
    drone.wait(1)

    drone.land()
```

Trace the branch choices

Stage	Requirement	Evidence
side = 0	Even branch	Long edge
side = 1	Odd branch	Short edge
side = 2	Even branch	Long edge
side = 3	Odd branch	Short edge

Extension missions

- Print whether each cycle is creating a long edge or a short edge.
- Reverse the branch values and explain whether the route is still a rectangle.
- Create a square by passing the same value for long_side and short_side.
- Rewrite the function using a list such as [60, 30, 60, 30], then compare readability.

Explain the logic

The conditional chooses the distance. The ninety-degree turn remains outside the branches because every corner uses the same angle.

ASSIGNMENT 03

Add direction choice to a pentagon function

A regular pentagon has five equal edges. Its exterior turn is 360 divided by 5, which is 72 degrees. This function adds one more decision: a Boolean parameter selects clockwise or counterclockwise rotation. The movement pattern remains the same; only the turn method changes.

FUNCTION ASSIGNMENT - BOOLEAN DIRECTION PARAMETER

```
def fly_pentagon(
    drone,
    side_length,
    turn_right
):
    drone.takeoff()
    drone.up(30)

    for side in range(__):
        drone.forward(side_length)

        if turn_right:
            drone.cw(__)
        else:
            drone.ccw(__)

    drone.wait(1)

    drone.land()
```

Required stages

Stage	Requirement	Evidence
Complete	Supply the repetition count and both turn angles.	No blanks remain
Call A	Use turn_right=True.	Clockwise plan
Call B	Use turn_right=False.	Counterclockwise plan
Compare	Explain what changes and what remains identical.	Four-sentence comparison

Extension missions

- Replace 72 with a named constant called PENTAGON_TURN.
- Add a sides parameter and calculate turn_angle = 360 // sides.
- Write a preview function that prints the route without calling EasyTello methods.
- Add a battery-ready function and call fly_pentagon only when it returns True.

No curve command

Keep every edge straight. The pentagon is produced by five forward movements separated by five rotations.

MISSION BOARD

Keep developing after the first solution works

Finishing the first working version is only the beginning. A developer should be able to predict the route, explain the control flow, revise inputs, and diagnose mistakes without immediately launching the aircraft. Use the stages below whenever a drone is unavailable or another team is completing a live test.

Stage	Requirement	Evidence
1. Pseudocode	Describe the route in plain language before Python.	Ordered steps
2. Diagram	Draw arrows and label every corner angle.	Route sketch
3. Trace	Record loop value, chosen branch, distance, and heading.	Trace table
4. Dry run	Replace movement methods temporarily with print statements.	Console preview
5. Review	Check indentation, arguments, takeoff count, and landing count.	Peer initials
6. Extension	Complete one additional mission from the assignment page.	Revised function

Debug by asking specific questions

- Does the loop repeat once for every intended edge?
- Is the turn inside the loop so every corner is created?
- Does each branch supply the intended distance or direction?
- Does the function call match the parameter order?
- Is there exactly one normal landing in the selected routine?
- Was the computer connected to the Tello Wi-Fi before the program began?

The complete idea

Functions name complete routes. Parameters make those routes adjustable. Loops create repeated edges and corners. Conditionals choose distances, directions, or entire routines. EasyTello sends only the methods Python actually reaches. The shape therefore begins as program structure before it becomes physical motion.

Final challenge

Design a regular hexagon function without `curve()`. Calculate its turn, write the loop, preview it with print statements, and justify every parameter.